
Document Design

Food Delivery Application

Academic Year 2020/21

Politecnico di Milano

Authors:

Livio Biondo

Francesco Gianchino

Giulia Valcamonica

1. Introduction

1.1 Scope

L'applicazione consiste in un frontend che accetta le richieste degli utenti e un backend che le elabora. Ci sono tre tipi di utenti che interagiscono con il servizio: (1) i clienti normali si registrano, fanno ordini e controllano lo stato degli ordini; (2) gli amministratori possono cambiare la disponibilità degli articoli; (3) i fattorini notificano le consegne riuscite. Il backend è decomposto in tre servizi, seguendo il paradigma dei microservizi: (1) il servizio utenti gestisce i dati personali degli utenti registrati; (2) il servizio ordini elabora e convalida gli ordini; (3) il servizio spedizioni gestisce la spedizione degli ordini validi. Alla ricezione di un nuovo ordine, il servizio ordini controlla se tutti gli articoli richiesti sono disponibili e, in caso affermativo, invia l'ordine al servizio spedizioni. Il servizio di spedizione controlla l'indirizzo dell'utente che ha effettuato

l'ordine e prepara la consegna.

I servizi non condividono lo stato, ma comunicano solo scambiando messaggi/eventi su argomenti Kafka.

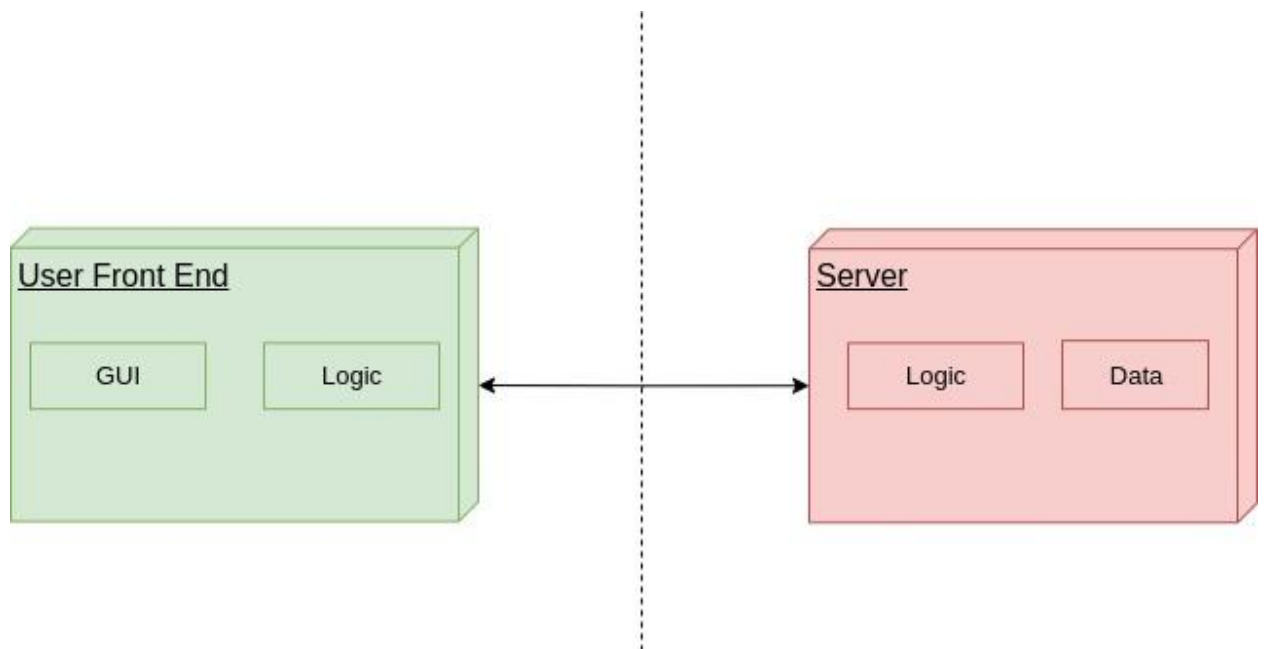
2. Architectural design

2.1 Overview

L'applicazione sarà caratterizzata da un'architettura standard client/server a 2 livelli. Entrambe le parti presenteranno un livello logico al fine di prendere decisioni in base all'input fornito.

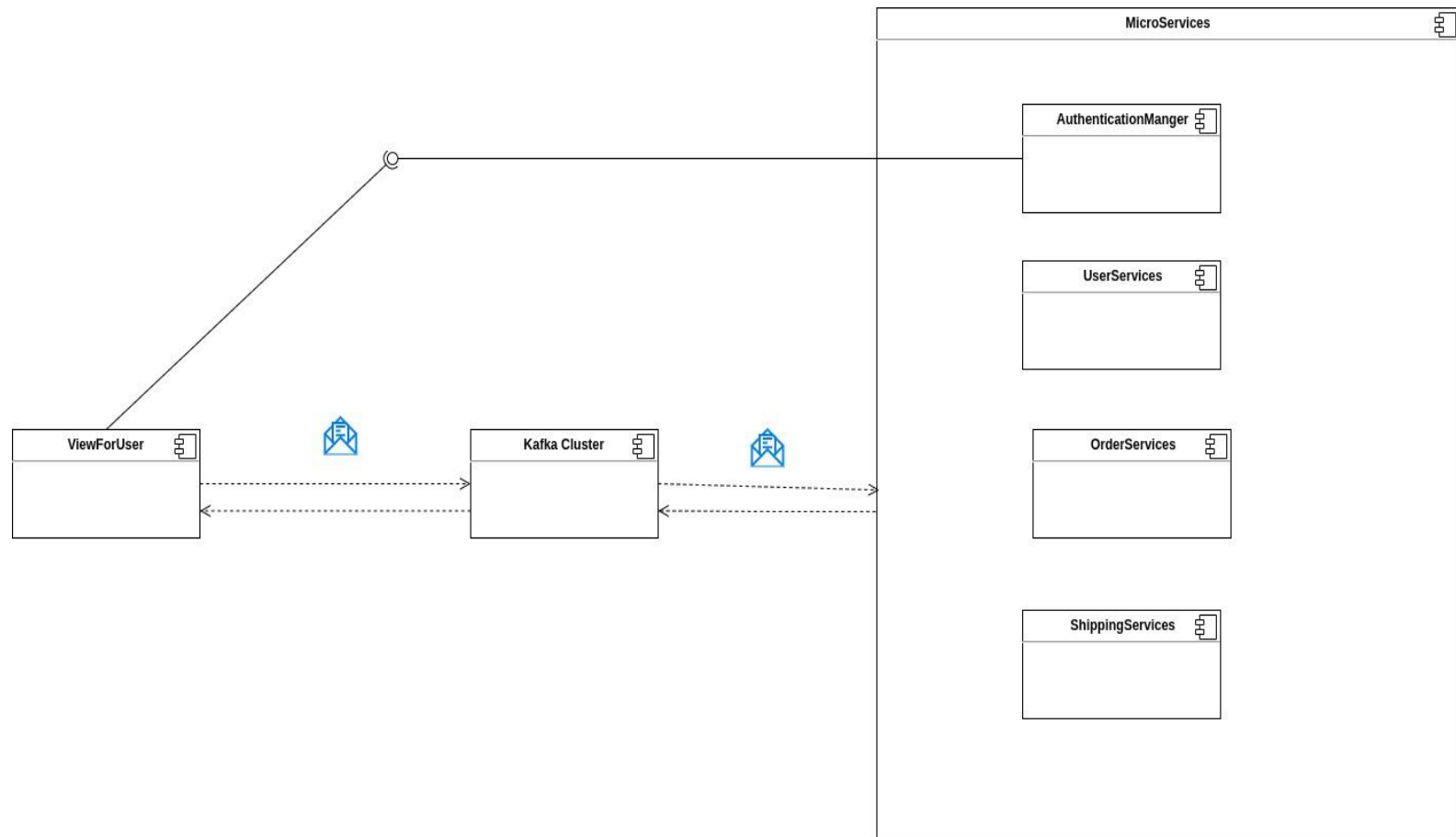
Il Front End host sul client sarà incaricato di presentare i dati all'utente e di gestire i suoi input attraverso una semplice GUI.

L'applicazione server si occuperà di recuperare il messaggio inviato dal Front End, gestire e memorizzare i dati in modo persistente.



2.2 Component View

L'applicazione segue i modelli architettonici dell'architettura event-driven e il paradigma dei microservizi, di conseguenza è composta da diversi macrocomponenti; di seguito viene mostrato un diagramma dei componenti con tutti i macrocomponenti del sistema e le loro relazioni.



ViewForUser: gestisce l'intera interfaccia grafica con cui vengono visualizzati i dati richiesti dall'utente. Gestisce l'input dell'utente e invia i relativi messaggi al cluster Kafka.

KafkaCluster: Contiene i broker Kafka che ricevono, salvano e permettono la lettura dei messaggi ai vari componenti dell'APP. Kafka assicura che ci siano anche diverse repliche dei messaggi in modo che il componente sia fault tolerant.

UserServices: Gestisce i dati personali degli utenti registrati e permette anche di creare l'account di tutti i tipi di utenti.

OrderServices: Elabora e convalida gli ordini.

ShippingServices: Gestisce la spedizione degli ordini validi.

Tutti e 3 i servizi si scambiano i dati necessari alle loro operazioni tramite messaggi kafka. Tutti i servizi hanno uno storage persistente dove si trovano i dati relativi alle loro funzionalità.

Ovviamente, i servizi leggono le richieste degli utenti e rispondono inviando le risposte.

AuthenticationManager: Gestisce le connessioni per il login degli utenti, riceve le credenziali dalla ViewForUsers, controlla se l'utente si è registrato e risponde alla view con il risultato dell'operazione.

2.3 Selected architectural styles and patterns

2.3.1 Two Tier Client-Server

L'applicazione avrà un'architettura multistrato:

- **Client:** è il livello responsabile della presentazione dei contenuti recuperati dall'Application Tier attraverso messaggi Kafka e della gestione degli input dell'utente.
- **Server:** questo è il livello in cui ci sono i broker, che memorizza i messaggi kafka, e i microservizi.

2.3.2 Microservices paradigm

L'intero server è strutturato come una collezione di servizi loosely-coupled che comunicano attraverso la rete. Questa scelta progettuale rende possibile avere un insieme di componenti che possono anche essere distribuiti su diverse macchine in diverse reti.

2.3.3 Event-Driven Architecture

Il componente centrale del server sono i broker Kafka che gestiscono la produzione, il rilevamento, il consumo e la reazione agli eventi (contenuti nei messaggi). Questi eventi prodotti e consumati dai servizi e dagli utenti permettono il trasferimento di informazioni e i cambiamenti nello stato del sistema.

2.3.4 Stateless Components

Ogni servizio non ha bisogno di memorizzare uno stato interno, tutti i dati sono memorizzati in modo persistente utilizzando alcuni file.

2.3.5 Loose Coupling

Il sistema sarà molto disaccoppiato: questa scelta minimizzerà le interazioni non necessarie tra i componenti e faciliterà il processo di manutenzione.

2.3.6 Thin Client

Il client presenta e gestisce l'input dell'utente, ma il suo esercizio è strettamente dipendente dal server per eseguire la maggior parte delle sue funzioni

2.4 Other design decisions

2.4.1 Apache Kafka

Per implementare i broker che gestiranno l'intera architettura event-driven, il software Apache Kafka e le sue API saranno utilizzati per creare gli argomenti, i produttori e i consumatori degli eventi.

3. Requirements traceability

3.1 Functional Requirements

- **R0:** Il sistema deve permettere agli utenti di registrarsi come clienti o amministratori o addetti alle consegne.
- **R1:** Il sistema deve permettere agli utenti di accedere con nickname e password.
- **R2:** Il sistema deve permettere ai clienti di fare ordini.
- **R3:** Il sistema deve permettere ai clienti di controllare lo stato dei loro ordini.
- **R4:** Il sistema deve permettere agli amministratori di cambiare la disponibilità delle merci.
- **R5:** Il sistema deve permettere ai fattorini di notificare le spedizioni riuscite.
- **R6:** Il sistema deve gestire i dati degli utenti.
- **R7:** Il sistema deve elaborare e convalidare gli ordini.
- **R8:** Per convalidare un ordine, il sistema deve controllare se gli articoli richiesti sono disponibili nella quantità desiderata.
- **R9:** Il sistema deve creare per ogni ordine valido la relativa spedizione.

3.2 Non-Functional Requirements

- **Manutenibilità:** Il modello di accoppiamento sciolto applicato a tutti i componenti del server permette una facile manutenzione, poiché i componenti sono molto indipendenti l'uno dall'altro. Inoltre, il componente Kafka garantisce flessibilità. In questo caso, poiché segue il pattern Event Based, se vengono aggiunte per caso nuove funzioni per i client, è sufficiente creare un nuovo Topic per i nuovi eventi e associarlo ad una funzione in un servizio specifico.
- **Tolleranza ai guasti:** I componenti di kafka permettono la replica degli argomenti e dei relativi eventi (messaggi) ridistribuiti tra i diversi broker attivi. Questo assicura che in caso di guasto o crash di uno o più broker il sistema continui a funzionare senza

interruzione del servizio e anche che, dato che i messaggi sono replicati, lo stato del sistema sia preservato. Nel nostro caso abbiamo una fabbrica di replicazione di 3 e quindi un cluster di 3 broker.